Node.js — Day 6 Complete Notes

Topic: Node.js Built-in Modules

Goal: Understand built-in modules, how to load them, the difference between local and built-in modules, and how module methods are used.

1. Built-in Modules

Node.js provides many modules as part of Node itself. These are called built-in modules. They can be used without installing them with npm.

```
const path = require("path");
```

You do not need to run npm install for Node.js built-in modules.

2. Common Built-in Modules

fs → work with files and directories.

path → work with file and folder paths.

os → information about the operating system.

http → create and work with HTTP servers.

events → work with events.

3. Local Module vs Built-in Module

Local module:

```
const math = require("./math");
```

The ./ tells Node to look for the module relative to the current file.

Built-in module:

```
const os = require("os");
```

A built-in module is loaded by its module name. You do not use ./.

4. ./ Means a Relative Path

./ means the current directory, so it is a relative path.

```
require("./math");
```

This means: find math relative to the current file.

../ means the parent directory and is also a relative path.

An absolute path identifies a location from the filesystem root, for example on macOS/Linux:

```
/Users/bhavana/project/math.js
```

5. Built-in Modules Have Methods

Importing a module gives you access to useful methods provided by that module.

path.basename() gets the final part of a path:

```
const path = require("path");
console.log(path.basename("/users/bhavana/app.js"));
```

```
// app.js
```

path.join() combines path parts:

```
const path = require("path");
console.log(path.join("users", "bhavana", "app.js"));
```

6. Practical path.join Example

```
const path = require("path");

const filePath = path.join("users", "bhavana", "notes.txt");

console.log(filePath);
```

The result is a path made from the individual parts:

```
users + bhavana + notes.txt
→ users/bhavana/notes.txt
```

path.join() combines path parts; it does not read the file or create a database.

7. Important Rules

• Built-in modules come with Node.js.

• Built-in modules do not normally need npm installation.

• Built-in modules are loaded by name, such as require("path").

• Local modules are loaded with a relative path, such as require("./math").

• ./ means the current directory and indicates a relative path.

• ../ means the parent directory and is also relative.

• Modules provide methods that perform useful tasks.

• path.join() combines path components.

8. Day 6 Quick Revision

fs: Files and directories.

path: File and folder paths.

os: Operating-system information.

http: HTTP servers.

events: Events.

./: Current directory; relative path.

../: Parent directory; relative path.

Absolute path: Starts from the filesystem root, such as /Users/... on macOS/Linux.

path.join(): Combines path parts.

Key Takeaway

Node.js includes useful built-in modules. Load built-in modules by name, while local modules use relative paths such as ./math.